Phase 1: Weeks 3 to 5 Goal: Deploy a runnable Android application capable of core scheduling operations using RAG for data retrieval.

1 What to do: Deploy a visual calendar interface rendering occupied nodes and suggested available slots for Android. (Frontend)

How to do: Utilize Android-native frameworks to build UI components that parse scheduling data arrays and render time blocks visually.

2 What to do: Enforce strict data privacy isolation preventing transmission of personal schedule data to unsecured public APIs. (Database)

How to do: Implement local encryption and restrict API calls to authenticated endpoints only, ensuring payload sanitization.

3 What to do: Implement basic username and password authentication protocols to secure initial access. (Backend)

How to do: Integrate a secure backend identity provider with hashed password storage and token-based session management.

4 Partial What to do: Develop a RAG ingestion module to index structured timetable data. (Backend)

How to do: Build a vector database pipeline to chunk data, generate embeddings, and store context for semantic retrieval, replacing static text parsing.

5 What to do: Engineer a deterministic availability engine capable of resolving one-semester schedules in under 200 milliseconds. (Backend)

How to do: Implement a time-complexity optimized search algorithm using interval trees to find free time slots rapidly.

6 What to do: Enforce absolute constraint satisfaction guaranteeing zero overlaps against fixed academic hard constraints. (Backend)

How to do: Implement a validation layer that rejects any scheduling request if the proposed time block intersects with pre-existing academic events retrieved via the RAG pipeline.

7 What to do: Develop a Natural Language Processing interface processing conversational scheduling queries mapping temporal constraints and task details. (AI and Server)

How to do: Integrate a natural language understanding model augmented with RAG to extract intents, entities, and context related to time duration and activity names.

8 What to do: Develop a conversational UI outputting scheduling rationale and executing explicit in-chat state confirmations for Accept, Reject, and Reschedule. (Frontend)

How to do: Create interactive chat components that render actionable buttons tied to backend scheduling state machine transitions.

9 What to do: Implement a graceful fallback protocol executing a manual UI selection state in under 1 second during AI timeouts or failures. (Frontend)

How to do: Set a strict timeout threshold on AI network requests and automatically route the user to a standard form-based interface if exceeded.

Phase 2: Weeks 6 to 11 Advanced NLP, RAG Edge Cases, and Telemetry Goal: Integrate advanced RAG parsing, handle conversational edge cases, and deploy system monitoring.

1 What to do: Engineer prompt pipelines achieving at least 98 percent adherence to structured JSON schemas with zero temporal hallucinations. (AI and Server)

How to do: Utilize few-shot prompting and rigorous JSON parsing validation rejecting and retrying malformed AI outputs.

2 Completion What to do: Expand the RAG pipeline to process unstructured timetable files including PDFs and images. (AI and Server)

How to do: Integrate document loaders and Optical Character Recognition to extract text from media files, chunk the content, generate embeddings, and update the vector index for retrieval.

3 What to do: Implement temporal ambiguity resolution logic for vague relative date inputs. (AI and Server)

How to do: Build a prompt layer that identifies missing context and asks the user clarifying questions or applies deterministic defaults based on current date.

4 What to do: Develop edge case resolution protocols handling continuous free block deficits via task splitting or priority demotion and managing explicit constraint conflicts. (Backend)

How to do: Implement a recursive scheduling algorithm that divides tasks into smaller chunks or drops low-priority items when continuous time is unavailable.

5 What to do: Implement priority classification algorithms utilizing user-assigned or AI-inferred qualifications. (Backend)

How to do: Add a priority weight integer to all task objects sorting the scheduling queue by this value before allocating time slots.

6 What to do: Implement error handling for non-diacritic Vietnamese, teencode, profanity, and unsupported special characters. (AI and Server)

How to do: Build a preprocessing pipeline utilizing dictionary mapping and regex filtering to normalize input text before RAG and Natural Language Processing execution.

7 What to do: Engineer a session state abandonment protocol handling unconfirmed interactions prior to new intent initiation. (Backend)

How to do: Implement a state machine that flushes pending scheduling confirmations if a user submits a new unrelated query.

8 Completion What to do: Implement secondary authentication protocols supporting Google OAuth and GitHub OAuth. (Backend)

How to do: Integrate OAuth2 client libraries and register application credentials with Google and GitHub developer consoles.

9 What to do: Develop an automated background notification system triggered prior to flagged events. (Backend)

How to do: Utilize Android AlarmManager or WorkManager APIs to register local push notifications based on the event timestamp.

10 What to do: Optimize AI inference execution on host hardware via a sequential request queue to maintain Time To First Token throughput achieving end-to-end latency under 4 seconds. (AI and Server)

How to do: Implement a First-In-First-Out request broker that serializes model inference calls to prevent Out Of Memory errors and context-switching overhead.

11 What to do: Implement IT telemetry logging pipeline latency, VRAM and RAM utilization, and queue depth. (Database)

How to do: Integrate a lightweight metrics collector that samples system resource usage and logs data points to a local or remote time-series database.

12 What to do: Deploy AI error taxonomy logs categorizing failed RAG retrievals and parsing attempts for prompt optimization. (AI and Server)

How to do: Catch retrieval and JSON parsing exceptions, logging the exact prompt, retrieved context, and failure reason to a persistent file for developer review.

13 What to do: Establish scheduler audit trails tracking background notification processes. (Database)

How to do: Write a logging service that records timestamps of all scheduled and executed notifications to ensure system reliability.